

Computer Architectures

Exam of 04.07.2023 - part II

Question 1

The aliquot sum $s(n)$ of a natural number n is the sum of all proper divisors of n , that is, all divisors of n other than n itself. For example, the aliquot sum of 28 is: $s(28) = 1 + 2 + 4 + 7 + 14 = 28$.

A natural number equal to its aliquot sum is called a *perfect number*. For example, 28 is a perfect number.

Two different natural numbers related in such a way that the aliquot sum of each is equal to the other number are called *amicable numbers*. For example, 220 and 284 are amicable numbers:

$$s(220) = 1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$$

$$s(284) = 1 + 2 + 4 + 71 + 142 = 220$$

Numbers whose aliquot sums form a periodic sequence are called *sociable numbers*. They are generalizations of the concepts of perfect numbers and amicable numbers. For example:

$$s(12496) = 1 + 2 + 4 + 8 + 11 + 16 + 22 + 44 + 71 + 88 + 142 + 176 + 284 + 568 + 781 + 1136 + 1562 + 3124 + 6248 = 14288$$

$$s(14288) = 1 + 2 + 4 + 8 + 16 + 19 + 38 + 47 + 76 + 94 + 152 + 188 + 304 + 376 + 752 + 893 + 1786 + 3572 + 7144 = 15472$$

$$s(15472) = 1 + 2 + 4 + 8 + 16 + 967 + 1934 + 3868 + 7736 = 14536$$

$$s(14536) = 1 + 2 + 4 + 8 + 23 + 46 + 79 + 92 + 158 + 184 + 316 + 632 + 1817 + 3634 + 7268 = 14264$$

$$s(14264) = 1 + 2 + 4 + 8 + 1783 + 3566 + 7132 = 12496.$$

You have to write the `isSociable` subroutine in ARM assembly language, which receives a natural number n in input, and repetitively computes the sequence of aliquot sums, until one of the following conditions occurs:

- the last computed aliquot sum is equal to the input parameter (i.e., the number is perfect, amicable, or sociable)
- the last computed aliquot sum is equal to 1 (i.e., the number is neither perfect, amicable, or sociable)
- 8 terms in the sequence have been computed (i.e., the number is neither perfect or amicable and probably is not sociable).

In the first case, the subroutine returns the length of the sequence. In the last two cases, it returns 0. For example:

- with $n = 28$, the subroutine computes $s(28) = 28$ and immediately stops returning 1
- with $n = 220$, the subroutine computes $s(220) = 284$, then $s(284) = 220$ and returns 2
- with $n = 12496$, the subroutine computes 5 aliquot sums and finally it returns 5
- with $n = 100$, it computes $s(100) = 117$, $s(117) = 65$, $s(65) = 19$, $s(19) = 1$. It stops returning 0.

In order to compute the aliquot sum of n , you have to use the following algorithm:

- 1) initialize $sum = 1$, $a = 2$
- 2) while(true):
- 3) if n is divisible by a :
- 4) $b = n / a$
- 5) if $a < b$:
- 6) $sum = sum + a + b$
- 7) else:
- 8) if $a == b$:
- 9) $sum = sum + a$
- 10) exit while and stop
- 11) $a = a + 1$

Computer Architectures

Exam of 04.07.2023 - part II

Question 2

Extend the project developed in the previous question as follows.

Declare an array of unsigned integers with the following values: {8128, 5564, 5400, 14264, 1305184, 1598470, 4938136}.

Initialize timer 1 to raise an interrupt every 2 seconds. In the interrupt handler, each time, a new value of the array is read (i.e., the first time the value 8128 is read; then 5564 and so on. After 7 interrupt calls, the 8th value is 8128 again). Then, the `isSociable` subroutine is called passing the current value of the array. The output of the subroutine is used to switch on the corresponding led (all other leds are switched off): value 1 -> led4; value 2 -> led 5; value 3 -> led6; ... ; value 8 -> led 11. If the subroutine returns 0, all leds are switched off.